

# Inheritance

# Introduction

---

- Inheritance is a fundamental OO concept
- Inheritance allows for well designed OO software
- Inheritance enables you to use polymorphism

# A Basic Example

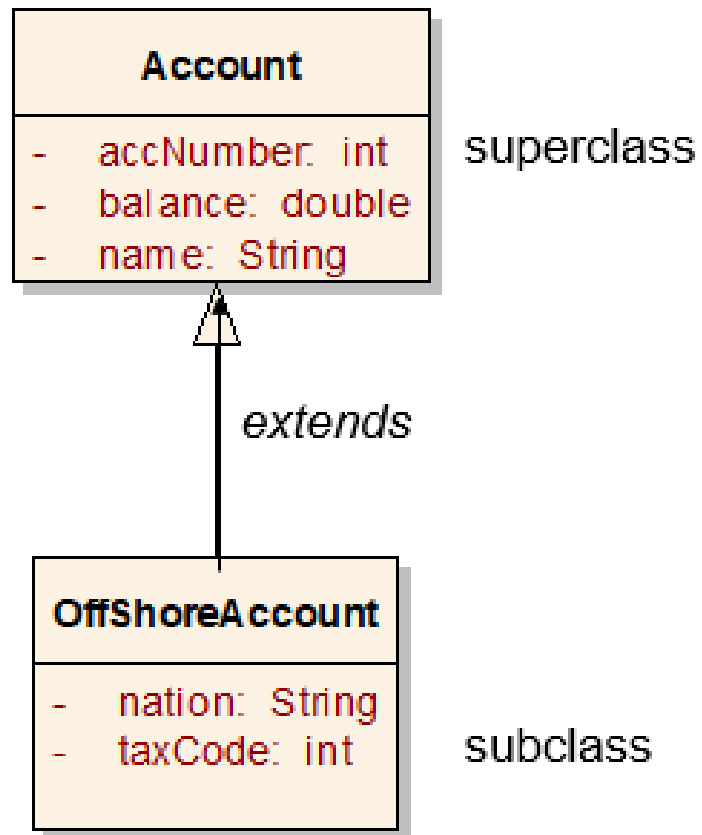
---

- The OffShoreAccount inherits everything from Account
- The keyword is *extends*

```
public class Account {  
    private double balance;  
    private String name;  
    private int accNumber;  
}
```

```
public class OffShoreAccount  
    extends Account {  
    private String nation;  
    private String taxCode;  
}
```

# Inheritance



```
public class Account {
    private double balance;
    private String name;
    private int accNumber;
}
```

```
public class OffShoreAccount
    extends Account {
    private String nation;
    private String taxCode;
}
```

# Questions

---

- What exactly is inherited?
- What about constructors?
- If the variables are private, can they be seen by subclasses?

# What is Inherited?

---

- Instance variables
- Instance methods
- Static variables
- Static methods
  
- Constructors are ***not*** inherited
- Variables and methods declared as private are not directly visible to subclasses

# Constructors

---

- Constructors are not inherited
- To set values of variables found in the superclass, call the superclass constructor from the subclass constructor

# The Superclass

---

- A normal class with a constructor

```
public class Account
{
    private double balance;
    private String name;

    // superclass constructor
    public Account(String name, double balance) {
        this.name = name;
        this.balance = balance;
    }
}
```



# The Subclass Constructor

---

- Note the call to the superclass constructor
- Use the keyword *super*
- Must be first line in constructor

```
public class OffShoreAccount extends Account {  
    private String nation;  
    private String taxCode;  
  
    // subclass constructor calls superclass constructor  
    public OffShoreAccount(String name, double balance, String n, String t) {  
        super (name, balance); // call to superclass constructor  
        this.nation = n;      // set up remaining variables  
        this.taxCode = t;  
    }  
}
```

# Creating Instances

---

- Call to the appropriate constructor

```
Account normal = new Account("Sarah", 300);  
OffShoreAccount off = new OffShoreAccount ("Nick", 20, "IsleOfMan", "NoTax");
```

# Visibility

---

- Variables declared as *private* are not visible to subclasses
  - Subclasses must therefore use an accessor method
- Variables in superclasses can be declared as *protected*
  - Subclasses can now see the property directly
  - BUT ... so can all other classes in the package



# The Four Levels of Visibility

---

- - **private** - only visible within that class
  - ~ **default** - visible to classes in the same package
  - # **protected** - visible to all subclasses *and all* classes in the same package
  - + **public** - visible to all classes
- 
- The symbols are the UML notations for the visibility

# Method Overriding

---

- Subclasses can **override** methods found in the superclass

```
public class Account {  
    protected double balance;  
    public void addInterest() {  
        balance *= interestRate;  
    }  
}
```

```
public class OffShoreAccount  
    extends Account {  
    public void addInterest() { // overriding  
        balance = balance * interestRate * taxFiddle;  
    }  
}
```

# Methods calling Superclass

---

- Methods can explicitly call superclass methods using the *super* keyword

```
public class Account {  
    protected double balance;  
    public void addInterest() {  
        balance *= interestRate;  
    }  
}
```

```
public class OffShoreAccount  
    extends Account {  
    public void addInterest() { // overriding  
        super.addInterest();  
        balance *= taxFiddle;  
    }  
}
```

# Using @Override

---

- When overriding a method, you could get the method name or signature wrong
  - No compiler checking is done
- **@Override** declares a method is overriding
  - If the method does not exist in a superclass, a compiler error will result

```
public class OffShoreAccount
    extends Account {
    @Override
    public void addInterest() {
        super.addInterest();
        balance *= taxFiddle;
    }
}
```

# The Object Class

---

- All classes in Java ultimately extend from Object
- Object is the immediate superclass of all classes that do not have a specified superclass

```
public class Account extends Object { // no need to specify this as it will
                                     // extend Object anyway
    ..
}
```



# Object Methods

---

- The Object class provides a number of methods including
  - Class getClass
  - boolean equals(Object obj)
  - int hashCode()
  - void wait() / void notify() / void notifyAll()
  - String toString()

# The toString() Method

---

- What appears in the console from the following line of code?

```
Account a = new Account("Mike", 50);  
System.out.println(a);
```

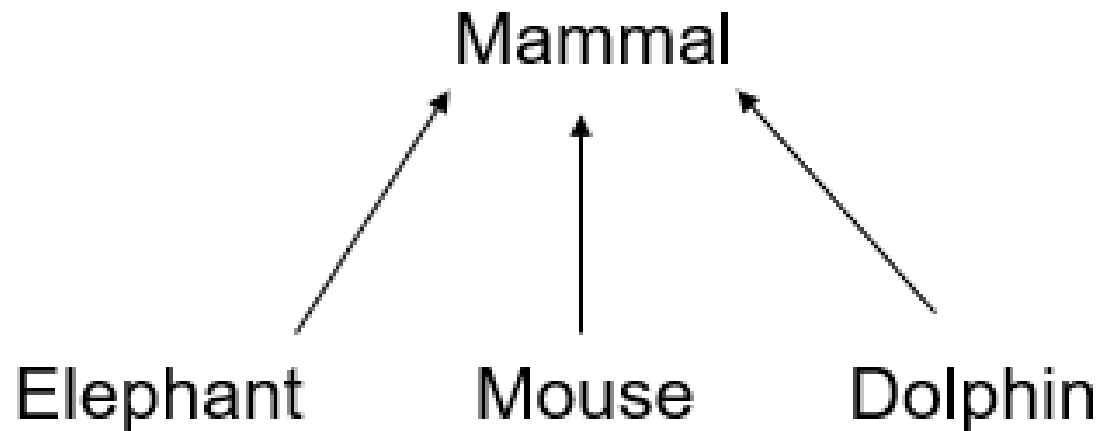
- The answer is a.toString()
  - Whenever an object reference appears where a String would go, the toString() method is called
- You can **override** the toString() method

```
public class Account {  
    ...  
    public String toString() {  
        return name + ":" + balance;  
    }  
}
```

# Abstraction

---

- Up until now, all the classes have been concrete classes
- What about **abstract** classes?



There is no such thing as a mammal. It is an abstract concept

# Abstract Classes

---

- Abstract classes are those that cannot be instantiated
- To define an abstract class, you must declare it as **abstract**

```
public abstract class Mammal {  
  
}
```

```
Mammal m = new Mammal(); // NOT ALLOWED
```

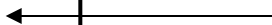
# Abstract Methods

---

- Abstract classes can force subclasses to have certain methods using abstract methods
  - Abstract methods are declared abstract
  - Abstract methods do not have a method body
  - Abstract method signatures ends with a semi-colon
- Abstract methods must be overridden by concrete subclasses

```
public abstract class Account {  
    public abstract void withdraw();  
}
```

Abstract method  
declaration (note the  
semi-colon)



# Subclassing Abstract Classes

---

- Concrete subclasses **must** override abstract methods
- Abstract subclasses do not have to override abstract methods

```
public abstract class Account {  
    public abstract void withdraw();  
}
```

```
public class SavingsAccount  
    extends Account {  
    public void withdraw() {  
        // must have this method  
    }  
}
```

# Polymorphism

---

- Examine the following statements
- Are they legal?

|   |                                                            |
|---|------------------------------------------------------------|
| ✓ | <code>Account acc1 = new Account();</code>                 |
| ✓ | <code>OffShoreAccount acc2 = new OffShoreAccount();</code> |
| ✓ | <code>Account acc3 = new OffShoreAccount();</code>         |
| ✓ | <code>Object acc4 = new Account();</code>                  |
| ✓ | <code>Object acc5 = new OffShoreAccount();</code>          |
| ✗ | <code>OffShoreAccount acc6 = new Account();</code>         |

# Polymorphism cont.

---

- The reason that most of those statements are legal is that
  - An OffShoreAccount **is a type of** Account
  - An Account **is a type of** Object
  - An OffShoreAccount **is a type of** Object
  - An Account **is not a type** of OffShoreAccount



# Which method?

---

- What about this code?

```
Account acc1 = new OffShoreAccount();  
acc1.addInterest();
```

- Which addInterest() method will be called?
- The declared type is Account, the actual type of the object is OffShoreAccount
- The method called will be that for the actual type of the object - *polymorphism*

# Arrays and Polymorphism (1)

---

- Within an array of Account references, there can be Accounts and OffShoreAccounts

```
Account[] array;           // declare the array
array = new Account[3];     // create the array
array[0] = new Account(); // populate the array
array[1] = new OffShoreAccount(); // include a subclass object
array[2] = new Account();
for (int i=0; i<array.length; i++) { // call a method found in both
    array[i].addInterest(); // class definitions
}
```

# Arrays and Polymorphism (2)

---

- The **declared** type of the array is Account
- The **runtime** type is Account or OffShoreAccount
- When the addInterest() method is called, the **runtime** type method is called - that is ***polymorphism***

# Preventing Inheritance

---

- Classes can be declared as final
  - They cannot be extended
- Methods can be declared as final
  - They cannot be overridden

```
public final class OffShoreAccount {  
    // cannot be subclassed  
}
```

```
public class Account {  
    public final void addInterest() {  
        // method cannot be overridden  
    }  
}
```

# Summary

---

- One class can inherit everything from another class except constructors
- A constructor in a subclass can call a constructor in a superclass
- Methods can be overridden
- Classes and methods can be abstract
- Polymorphism - the declared type can be different from the runtime type
- Classes and methods can be declared as final